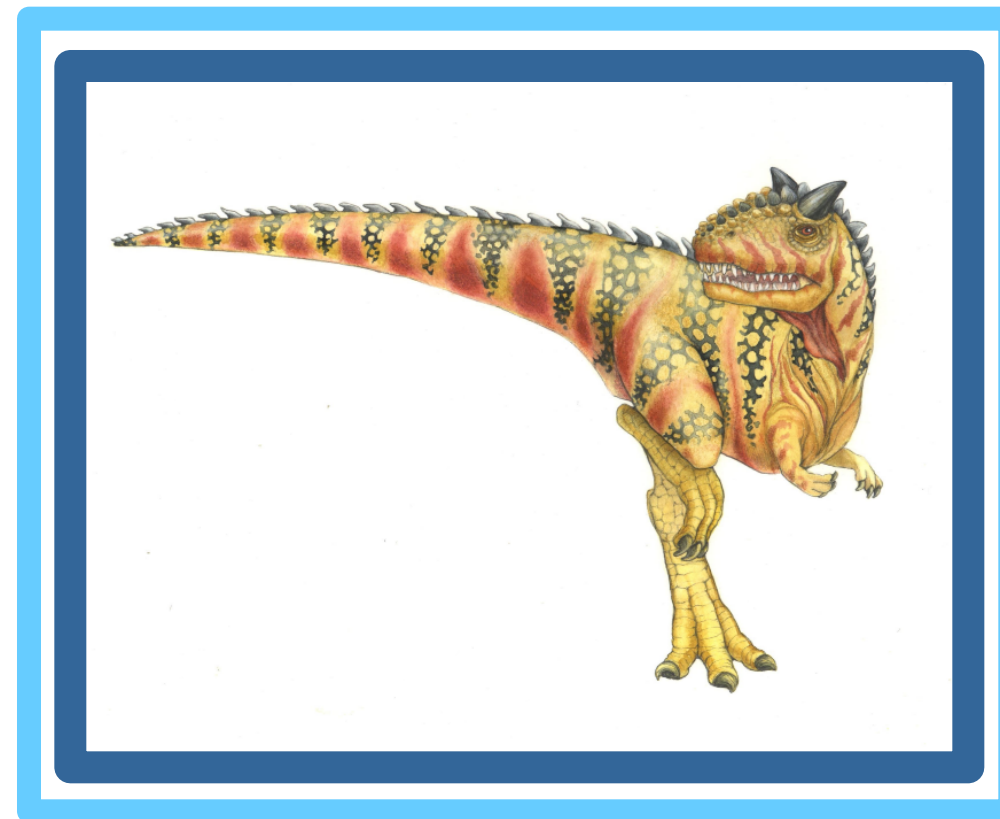
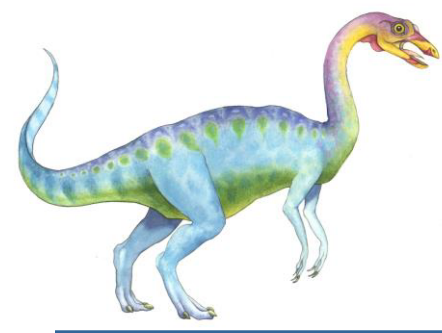


Chapter 8: Deadlocks





Chapter 8: Deadlocks

- ☐ System Model
- ☐ Deadlock Characterization
- ☐ Methods for Handling Deadlocks
- ☐ Deadlock Prevention
- ☐ Deadlock Avoidance
- ☐ Deadlock Detection
- ☐ Recovery from Deadlock





Methods for Handling Deadlocks

- Ensure that the system will *never* enter a deadlock state:
 - Deadlock prevention
 - Deadlock avoidance
- Allow the system to enter a deadlock state and then recover
- Ignore the problem and pretend that deadlocks never occur in the system; used by most operating systems, including UNIX

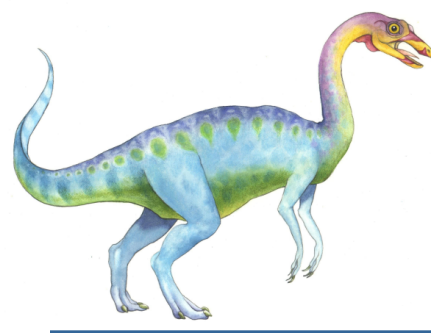




Avoidance Algorithms

- Single instance of a resource type
 - Use a resource-allocation graph
- Multiple instances of a resource type
 - Use the banker's algorithm

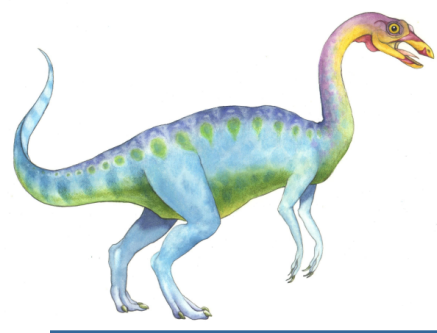




Banker's Algorithm

- Multiple instances
- We need prior information to avoid deadlock
- What kind of information we are looking for:
 - What is the max number of instances of a resource can a process request?
 - How many instances of a resource each process currently hold?
 - How many instances are available in the system?
- When a process requests a resource, it may have to wait
- When a process gets all its resources it must return them in a finite amount of time





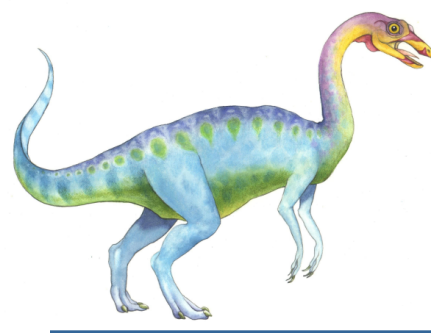
Example: P_1 Request (1,1,0,0)

- Need matrix?
- Safe sequence? Total instances for each resource type $\rightarrow A=12$ $B=12$ $C=8$ $D=10$

	Allocation				Max				Available			
	A	B	C	D	A	B	C	D	A	B	C	D
P0	2	0	0	1	4	2	1	2	3	3	2	1
P1	3	1	2	1	5	2	5	2				
P2	2	1	0	3	2	3	1	6				
P3	1	3	1	2	1	4	2	4				
P4	1	4	3	2	3	6	6	5				

- Safe sequence $\rightarrow P0$ $P3$ $P4$ $P1$ $P2$





Resource-Request Algorithm for Process P_i

$Request_i$ = request vector for process P_i . If **$Request_i[j] = k$** then process P_i wants k instances of resource type R_j

1. If **$Request_i \leq Need_i$** go to step 2. Otherwise, raise error condition, since process has exceeded its maximum claim
2. If **$Request_i \leq Available$** , go to step 3. Otherwise P_i must wait, since resources are not available
3. Pretend to allocate requested resources to P_i by modifying the state as follows:

$Available = Available - Request_i$;

$Allocation_i = Allocation_i + Request_i$;

$Need_i = Need_i - Request_i$;

- If safe $\Rightarrow$ the resources are allocated to P_i
- If unsafe $\Rightarrow P_i$ must wait, and the old resource-allocation state is restored





Avoidance Algorithms

- Single instance of a resource type
 - Use a resource-allocation graph
- Multiple instances of a resource type
 - Use the banker's algorithm





Resource-Allocation Graph

A set of vertices V and a set of edges E .

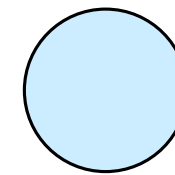
- V is partitioned into two types:
 - $P = \{P_1, P_2, \dots, P_n\}$, the set consisting of all the processes in the system
 - $R = \{R_1, R_2, \dots, R_m\}$, the set consisting of all resource types in the system
- **request edge** – directed edge $P_i \rightarrow R_j$
- **assignment edge** – directed edge $R_j \rightarrow P_i$



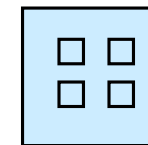


Resource-Allocation Graph (Cont.)

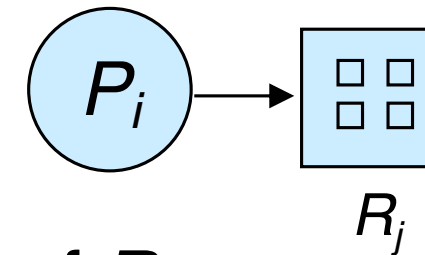
- Process



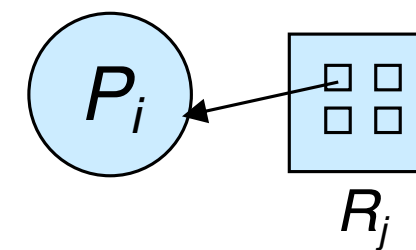
- Resource Type with 4 instances



- P_i requests instance of R_j

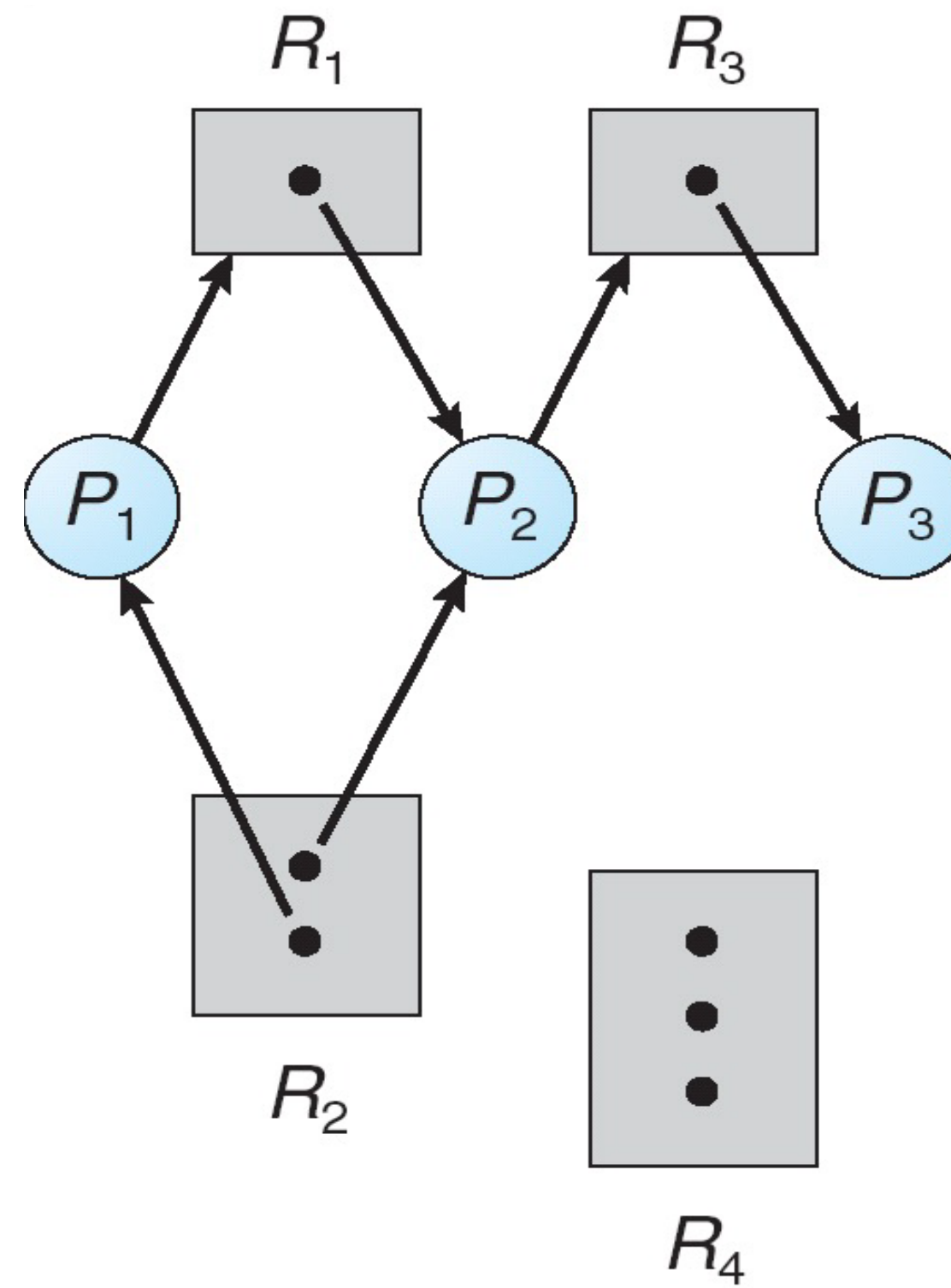


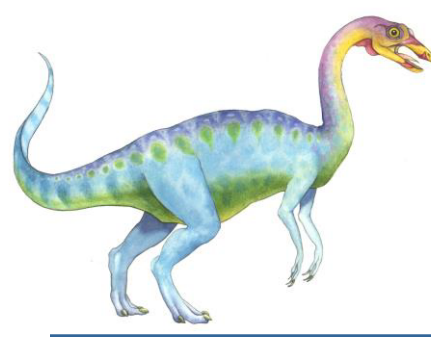
- P_i is holding an instance of R_j



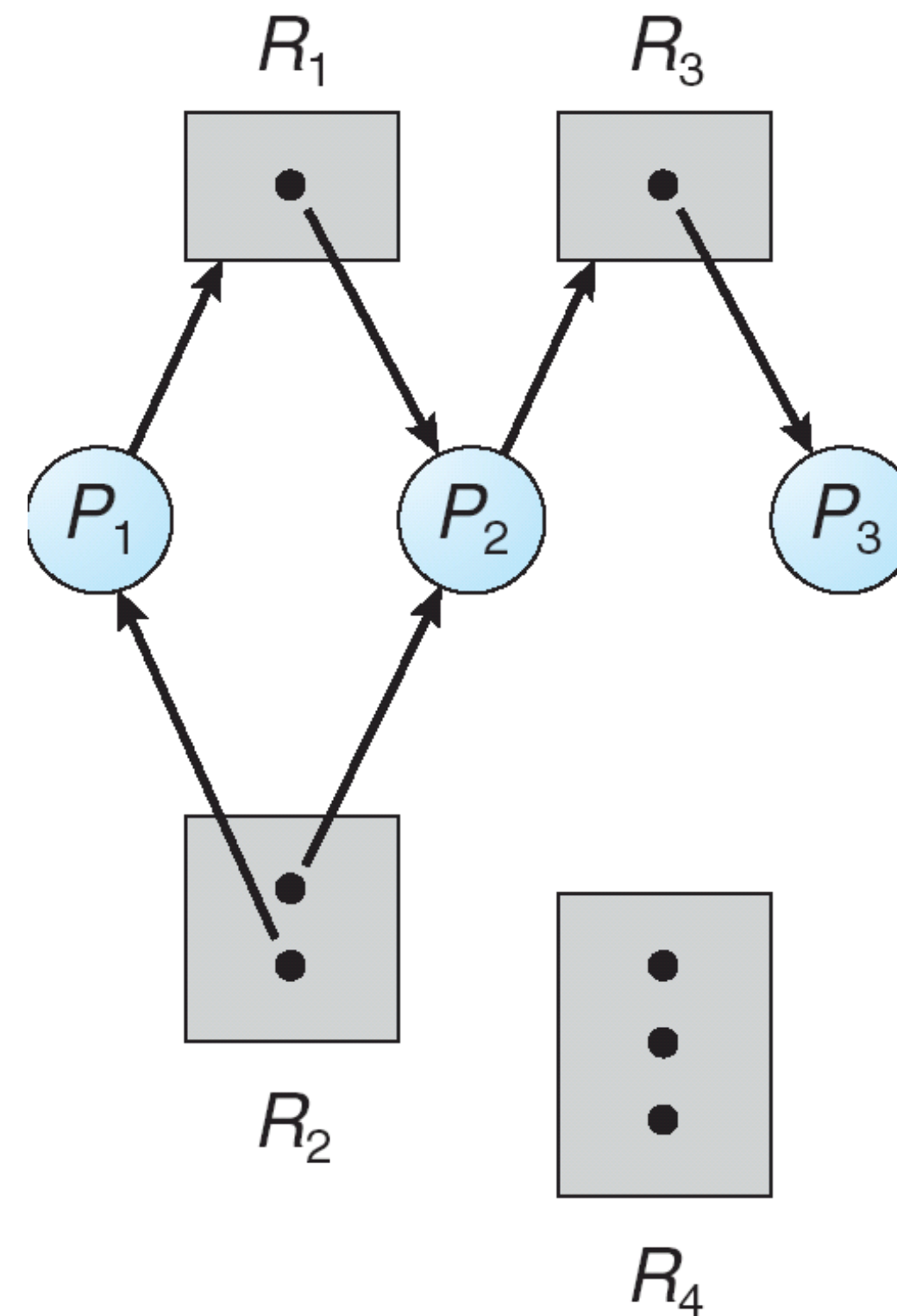


Example of a Resource Allocation Graph





Example of a Resource Allocation Graph



What are V?

$P = \{p_1, p_2, p_3\}$

$R = \{R_1, R_2, R_3, R_4\}$

Edges?

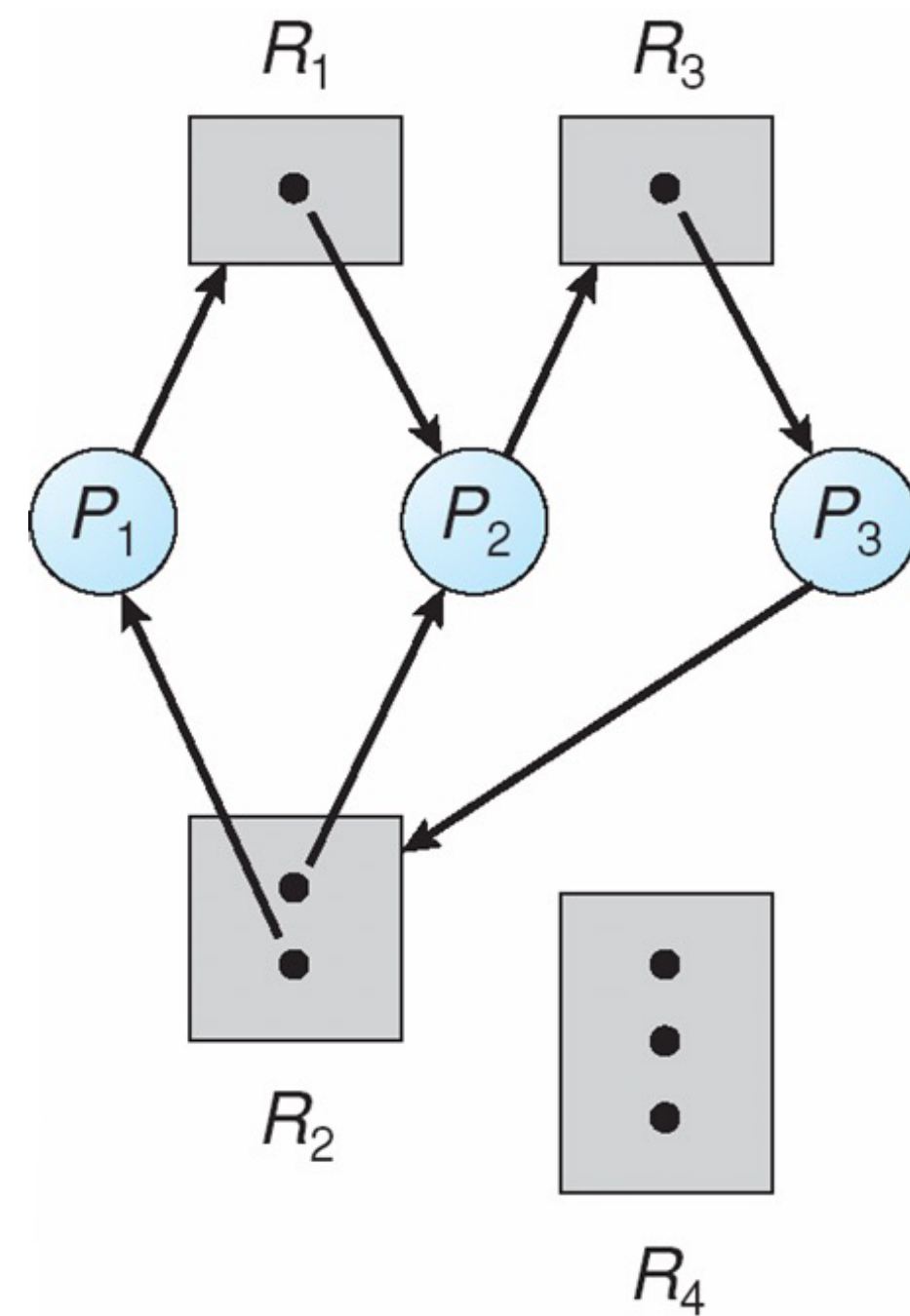
Request Edge = $\{p_1 \rightarrow R_1, p_2 \rightarrow R_3\}$

Assignment edges = $\{R_1 \rightarrow p_2, R_2 \rightarrow p_2, R_2 \rightarrow p_1, R_3 \rightarrow p_3\}$





Resource Allocation Graph With A Deadlock



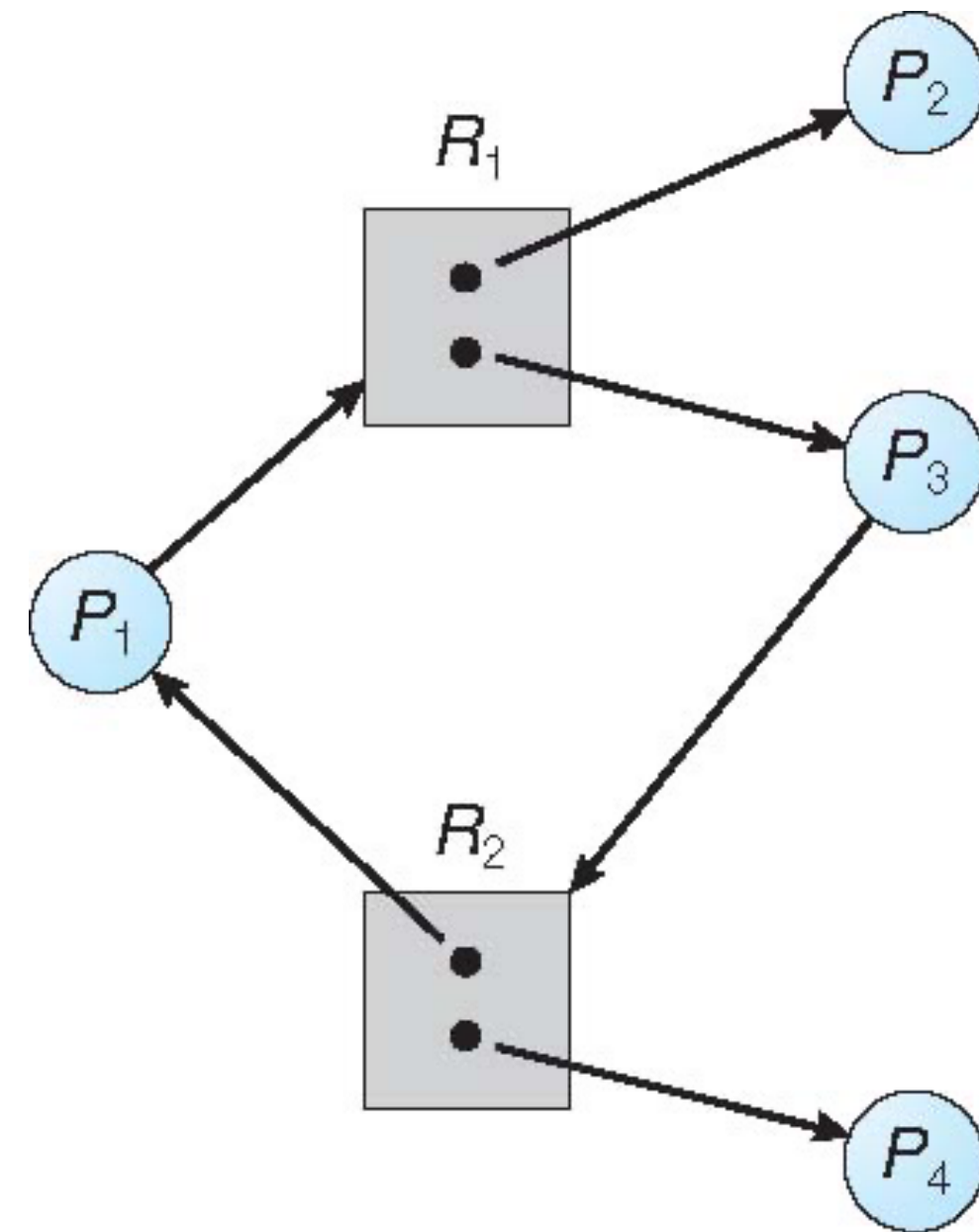
Allocation Matrix	R1	R2	R3
P1	0	1	0
p2	1	1	0
p3	0	0	1

Request or Need matrix	R1	R2	R3
P1	1	0	0
p2	0	0	1
p3	0	1	0





Graph With A Cycle But No Deadlock



Allocation Matrix	R1	R2
P1	0	1
p2	1	0
p3	1	0
P4	0	1

Request or Need matrix	R1	R2
P1	1	0
p2	0	0
p3	0	1
p4	0	0

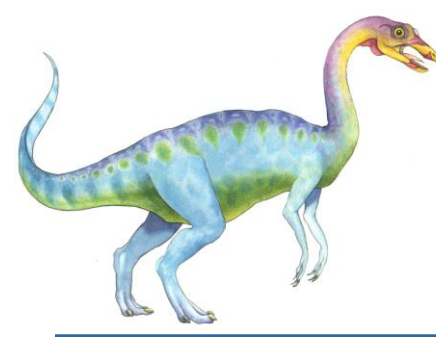




Basic Facts

- If graph contains no cycles $\Rightarrow$ no deadlock
- If graph contains a cycle $\Rightarrow$
 - if only one instance per resource type, then deadlock
 - if several instances per resource type, possibility of deadlock

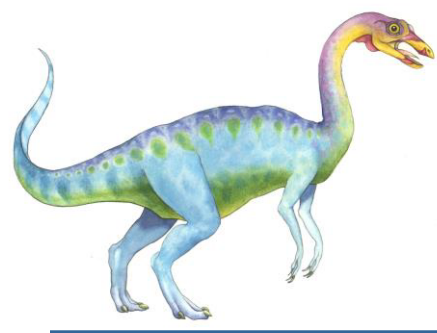




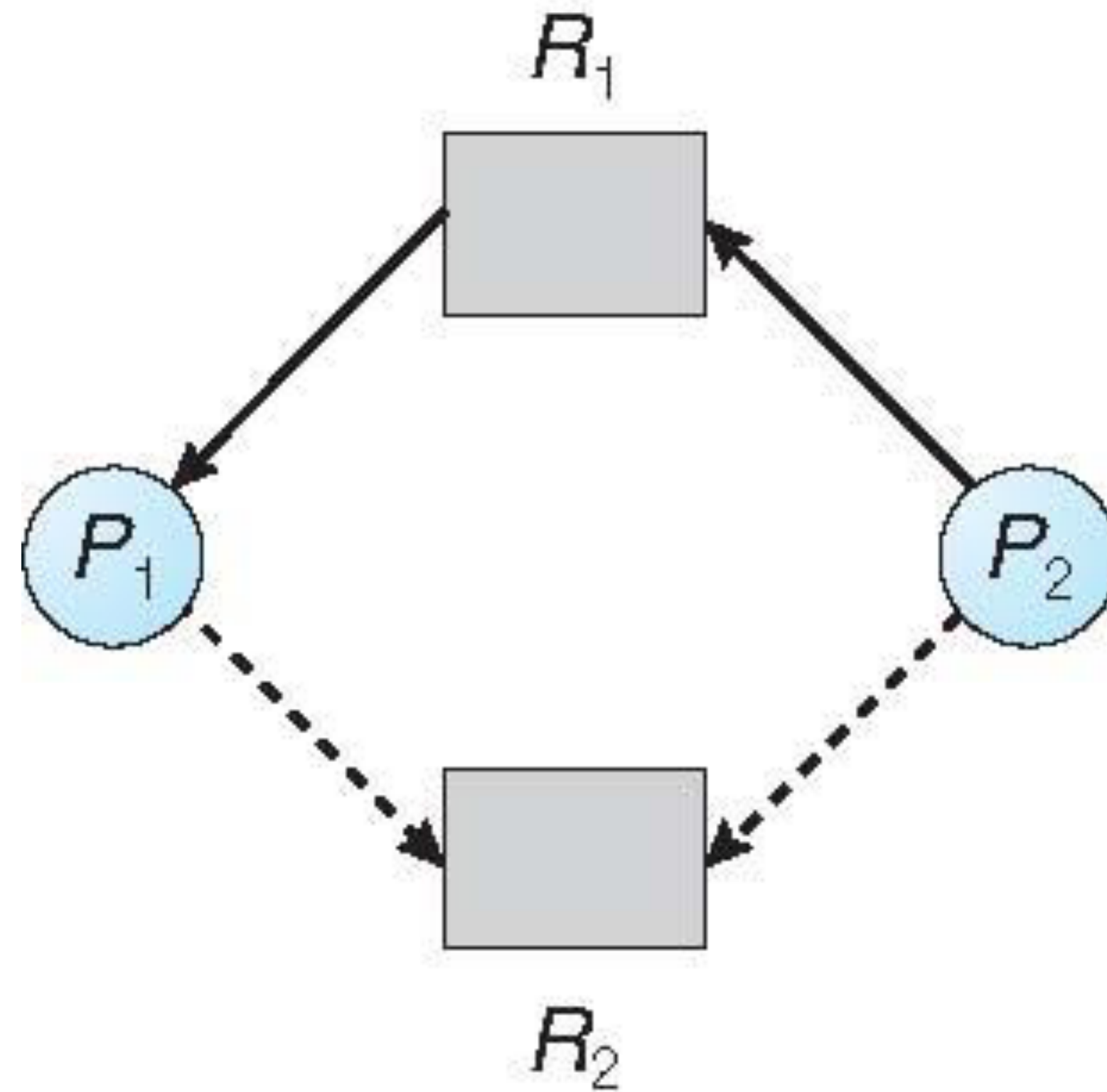
Resource-Allocation Graph Scheme

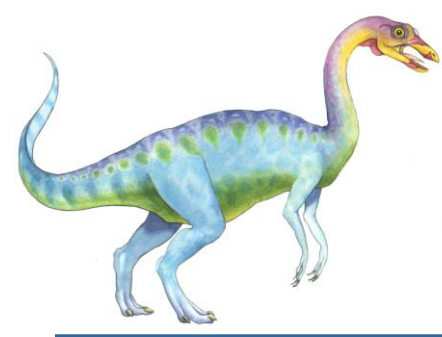
- **Claim edge** $P_i \rightarrow R_j$ indicated that process P_j may request resource R_j ; represented by a dashed line
- Claim edge converts to request edge when a process requests a resource
- Request edge converted to an assignment edge when the resource is allocated to the process
- When a resource is released by a process, assignment edge reconverts to a claim edge
- Resources must be claimed *a priori* in the system
- We use this algorithm only **when all resources have only one instance.**



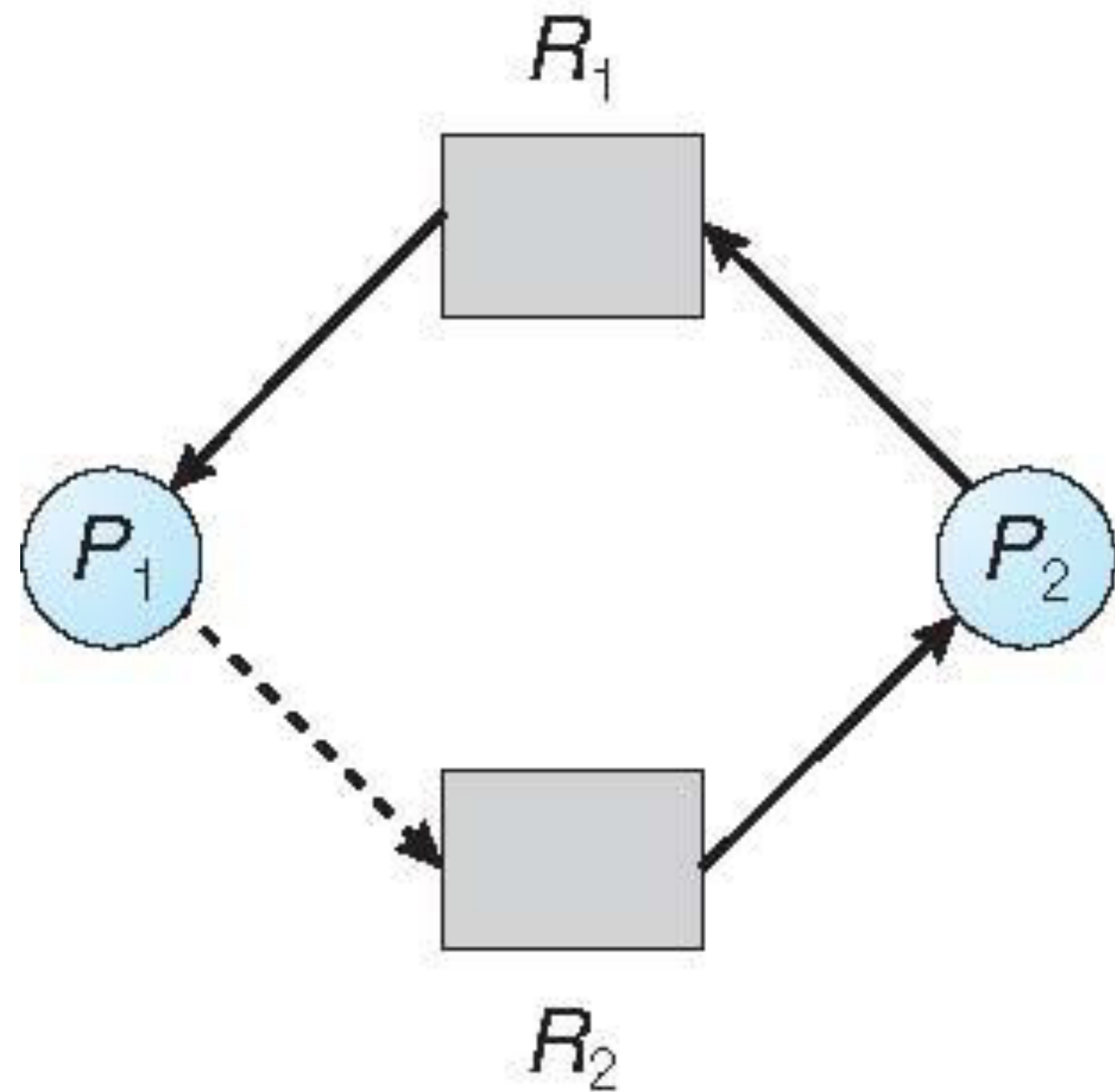


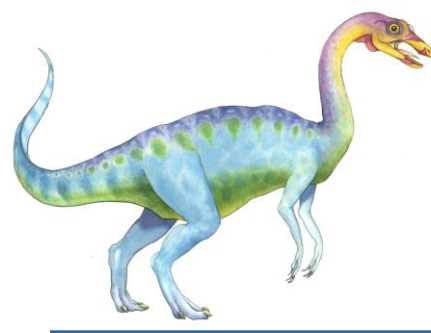
Resource-Allocation Graph





Unsafe State In Resource-Allocation Graph

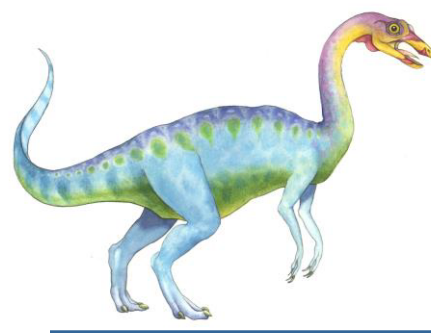




Deadlock Prevention

- **Removal of Mutual Exclusion** – not required for sharable resources (e.g., read-only files); must hold for non-sharable resources
 - Make all resources sharable
 - It is not practical
- **Removal **Hold and Wait**** – must guarantee that whenever a process requests a resource, it does not hold any other resources
 - Require process to request and be allocated all its resources before it begins execution, or allow process to request resources only when the process has none allocated to it.
- **Low resource utilization; starvation possible**
 - Not practical





Deadlock Prevention (Cont.)

- **Removal of No Preemption –**
 - If a process that is holding some resources requests another resource that cannot be immediately allocated to it, then all resources currently being held are released
 - Preempted resources are added to the list of resources for which the process is waiting
 - Process will be restarted only when it can regain its old resources, as well as the new ones that it is requesting
 - Can be done only for higher priority processes
- **Removal of Circular Wait –** impose a total ordering of all resource types, and require that each process requests resources in an increasing order of enumeration
 - Example: different resources and a process can request only those resources with higher number





Chapter 8: Deadlocks

System Model

Deadlock Characterization

Methods for Handling Deadlocks

Deadlock Prevention

Deadlock Avoidance

Deadlock Detection

Recovery from Deadlock



Quiz

1. What are the 4 approaches to handle deadlocks?
2. What are the two approaches that ensure the system never enters a deadlock state?
3. If a graph contains a cycle, there is definitely a deadlock present — T or F?
4. What are the two sets of edges in a resource allocation graph?
5. Resource allocation graph based algorithms are used when there is multiple instances of resource type is available — T or F?
6. Claim edge converts to an assignment edge when a process requests a resource and to a request edge when the resource is allocated to the process — T or F?



Chapter 8: Deadlocks

System Model
Deadlock Characterization
Methods for Handling Deadlocks
Deadlock Prevention
Deadlock Avoidance
Deadlock Detection
Recovery from Deadlock

